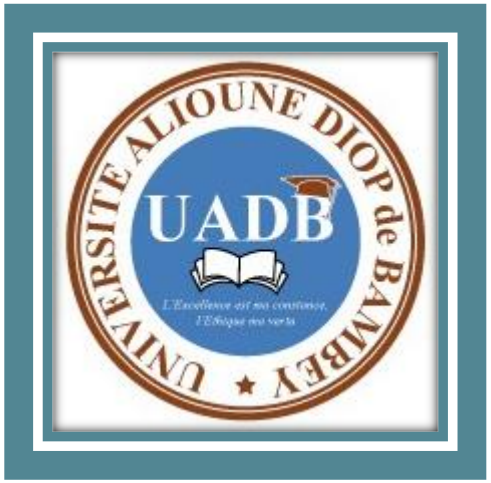
UNIVERSITE ALIOUNE DIOP DE BAMBEY(UADB) SATIC/TIC/WEB3



THREAD

Mme. KASSE

Définition

- En java, il est possible de simuler l'exécution de plusieurs programmes en même temps . C'est le multithreading. L'exécution de chaque programme est alors appelée un thread.
- Processus vs *threads (= processus légers)*
  - processus : espaces mémoire séparés
  - *threads : espace mémoire partagé*
- \+ efficace
- \- robuste
  - • le plantage d'un *thread peut perturber les autres*
  - • le plantage d'un processus n'a pas (normalement) d'incidence sur les autres
- Les threads sont donc des traitements qui vivent au sein d'un même processus.

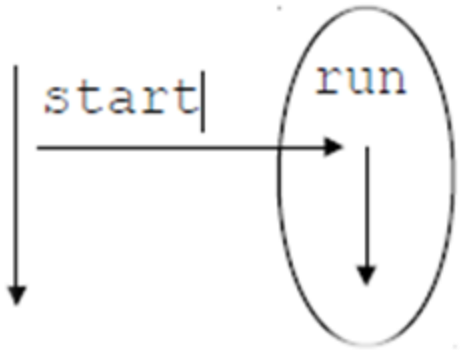
Création de thread

- Plusieurs solutions pour l'écriture d'une classe thread
  - • Classe héritant de java.lang.Thread
  - • Classe implantant l'interface java.lang.Runnable

```
interface Runnable {
    public void run();
}
```

Lancement d'un thread

- Pour lancer un thread on fait appel à la méthode **start().**
- Pour définir ce que fera un thread il faut redéfinir la méthode **run()**.



- **Remarques :**
- Création d'autant de threads que nécessaire.
- Un thread meurt lorsque sa méthode **run()** se termine

Lancement d'un thread

- **ATTENTION :**

- L 'appel de la méthode **start()** est fait **une et une seule fois** pour chaque ***thread.***

- On appelle jamais directement **run()**. **(start()** le fait**)**

- Le code de la méthode **run()** est exécuté dans le thread courant.

Lancement d'un thread

- En Java le multithreading signifie qu'il faut
  - créer un thread,
  - écrire la tâche exécutée par le thread,
  - lier la tâche au thread.
- Pour réaliser cela, il faut étudier et comprendre la classe Thread.

Constructeur et Méhodes de thread

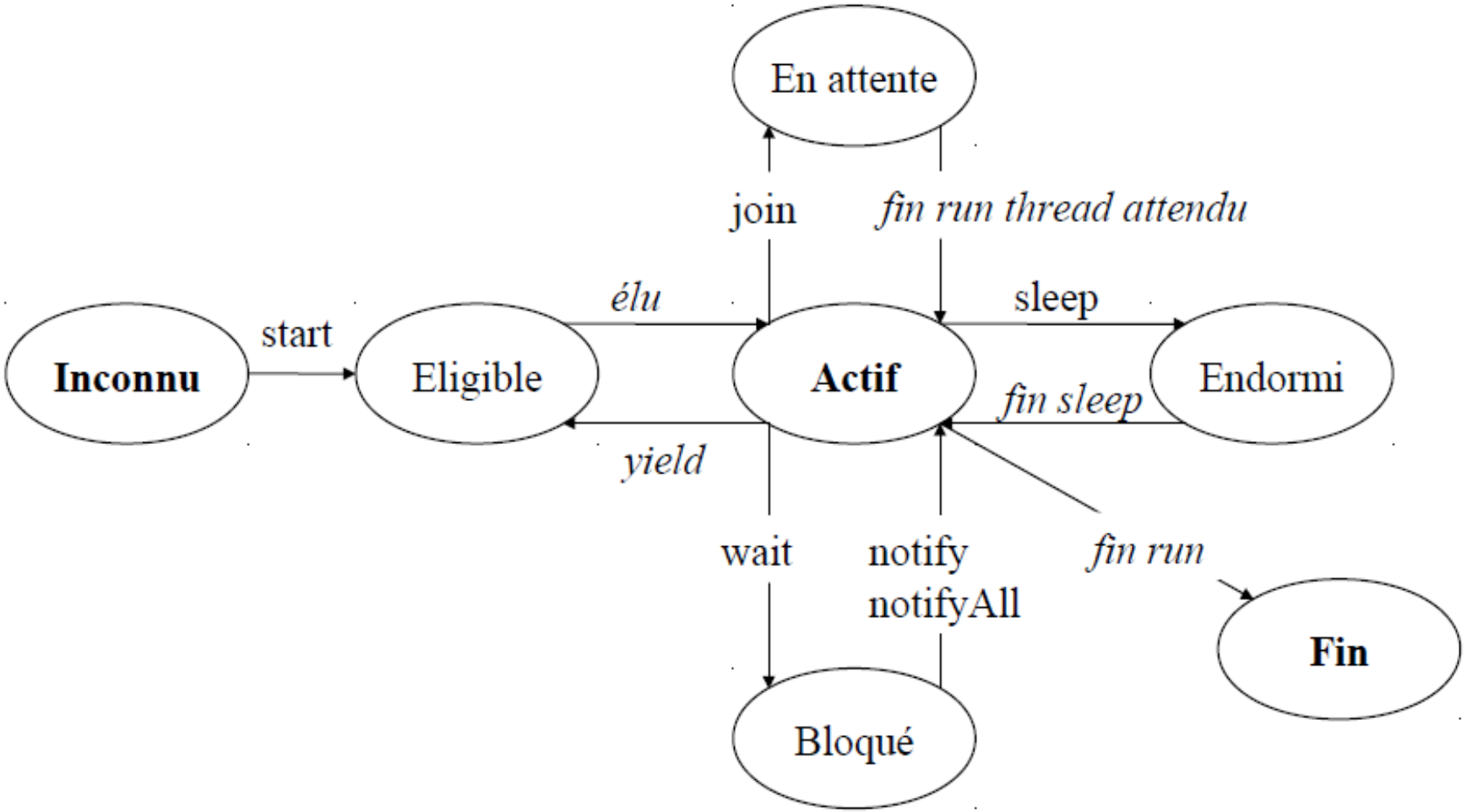
THREAD	Constructeur(public)	Méthode	Fonct.Méthode
	Thread()	void distroy()	Arrêt brutal du thread
	Thread(Runnable r)	Int getPriority()	Renvoie la priorité du thread
	Thread(Runnable r, String name)	void setPriority(int priority)	Change la priorité du thread
	Thread(String name)	String getName()	Retourne le nom du thread
		Void interrupt()	Pour interrompre le thread
		Static boolean interrupt()	Teste si le thread courant est interrompu
		void join()	Attendre la mort du thread
		Void resume()	Redémarre le thread
		Void run()	Contient le code à exécuter par le thread

Constructeur et Méhodes de thread

THREAD	Méthode	Fonction de la Méthode
	static void sleep(long m)	Mettre en veille le thread
	void start()	Démarre l' exécution du thread
	String toString()	Renvoie le nom du thread, sa priorité et le groupe auquel il appartient
	boolean isAlive()	Indique si le thread est actif ou non
	Threadgroup getThreadGroup()	Renvoie un objet qui encapsule le groupe auquel le thread appartient
	Void yiel()	Indique que le thread peut être suspendu au profits des autres

Cycle de vie d'un thread

- **Résumé du cycle de vie d'un thread:**



Exécuter plusieurs thread simultanément

Héritant de la classe Thread

```
public class MonThread extends Thread {
public void run() {
for (int i = 1; i <= 5; i++)
System.out.println(i + " " + getName());
}
public static void main (String[] args) {
Thread t1=new MonThread();
t1.start();
Thread t2=new MonThread()
t2.start();
}
}
```

```
public class MonThread1
extends Thread {
static int  j = 1;
public void run() {
for (int i = 1; i <= 8; i++) {
System.out.println(getName()+" "+j);
j++;}
}
public static void main (String[] args) {
new MonThread1().start();
new MonThread1().start();
}
}
```

```
Thread-1  1
Thread-1  2
Thread-1  3
Thread-0  3
Thread-0  4
Thread-0  5
Thread-0  6
Thread-0  7
Thread-0  8
Thread-0  9
Thread-0  10
Thread-1  12
Thread-1  13
Thread-1  14
Thread-1  15
Thread-1  16
```

Exécuter plusieurs thread simultanément

Implémentant l'interface Runnable

```
public class MonThread2 implements Runnable {
public void run() {
for (int i = 1; i <= 5; i++)
System.out.println(i + " "+Thread.currentThread().getName());
}
public static void main (String[] args) {
Runnable r= new MonThread2();
Thread t1=new Thread(r,"A");
    t1.start();
Thread t2=new Thread(r,"B");
    t2.start();
}
}
```

Exécuter plusieurs thread simultanément

Utilisation de la méthode sleep

```
class A extends Thread {
public void run() {
for(int i=0; i<8; i++) {
System.out.print("A"+i+" ");
try {sleep(100);} catch
(InterruptedException e) {}
}
}

class B extends Thread {
public void run() {
for(int i=0; i<8; i++) {
System.out.print("B"+i+" ");
try {sleep(200);} catch
(InterruptedException e) {}
}
}
```

```
public class TestThread1 {
public static void main(String args[]) {
new A().start(); new B().start();
}
```

Exclusion et moniteurs: Synchronized

- Pour que plusieurs processus puissent se partager sainement des objets afin de coopérer ou d'utiliser des ressources, il faut pouvoir limiter le parallélisme en s 'assurant qu'un certain objet ne subisse pas en même temps plusieurs séquences d'actions :

```
class Impr extends Thread {
String txt;
public Impr(String txt){this.txt=txt;}
public void run() {
for(int j=0; j<2; j++) {
for(int i=0; i<txt.length(); i++) {
try {sleep(100);} catch
(InterruptedException e) {};
System.out.print(txt.charAt(i));
}
}
```

```
public class TestThread3 {
 public void main(String argv[]) {
new Impr("BONJOUR ").start();
new Impr("AU REVOIR ").start();
}
}
```

Exclusion: Synchronized

- Pour assurer les exclusions, Java offre la primitive **synchronized.** En Java *tout objet est susceptible d'être un motif d'exclusion. La syntaxe générale d'une exclusion* relative à un objet obj est :
- synchronized(obj) { *bloc d'instructions }*
- On peut alors programmer comme suit l'exclusion souhaitée :

- class Exclusion {};

Exclusion: Synchronized

```
class ImpSync  extends Thread {
String txt;
 Exclusion excImp = new Exclusion();
public ImpSync(String txt){this.txt=txt;}
public void run() {
for(int j=0; j<2; j++) {
synchronized(excImp){
for(int i=0; i<txt.length(); i++) {
try {sleep(100);} catch(InterruptedException e) {}
System.out.print(txt.charAt(i));
}
System.out.print(" ");
}}}}
```

Moniteurs: Synchronized

- L'exclusion peut porter sur une méthode complète. La syntaxe est dans ce cas :

```
class A {
 ...
synchronized ... P(...) { ... }
 ...
 }

 class MoniteurImpression {
 synchronized void imprTexte(String txt) {
 for(int i=0; i<txt.length(); i++) {
 try {Thread.sleep(100);}
 catch(InterruptedException e) {}
 System.out.print(txt.charAt(i));
 }
 }
 }
```

Moniteurs: Synchronized

```
class ImprMonit extends Thread {
String txt;
 MoniteurImpression m1 = new MoniteurImpression();
public ImprMonit(String t){txt=t;}
public void run() {
for(int j=0; j<2; j++) { m1.imprTexte(txt);};
}
}
```

*Attente explicite : wait - notify

- Attente qu'une condition soit satisfaite : **wait()**
- À l'intérieur d'un moniteur (c'est à dire d'une méthode ou d'un bloc qualifié par synchronized), un processus peut se mettre en attente au moyen de **wait().**
- Cette primitive est susceptible de déclencher, pendant l'attente, une exception de type **InterruptedException**, ce qui oblige souvent à utiliser la forme :

  try {wait();} catch(InterruptedException e) {...}

- Réveil des processus : **notify() et notifyAll()**
- Un processus sort de l'attente lorsqu'un autre processus exécute notify() au sein de ce même moniteur. Cette primitive existe sous deux formes :
- **notify() :** relance *un processus en attente dans ce moniteur*,
- **notifyAll():** relance *tous les processus en attente dans ce moniteur.*

*Attente explicite : wait - notify

L'exemple suivant illustre la programmation du classique tampon producteur-consommateur.

```
class MoniteurProdCons {
String tampon; boolean estVide=true;
synchronized void prod(String m) {
if(!estVide){ System.out.println("PRODUCTEUR ATTEND");
try {wait();} catch(InterruptedException e) {};
}
System.out.println("PRODUIT : " + m);
tampon=m; estVide=false; notify();
}
synchronized String cons() {
if(estVide){ System.out.println("CONSOMMATEUR ATTEND");
try {wait();} catch(InterruptedException e) {};
}
System.out.println("CONSOMME : " + tampon);
String resul=tampon; estVide=true; notify();
return resul;
}
}
```

*Attente explicite : wait - notify

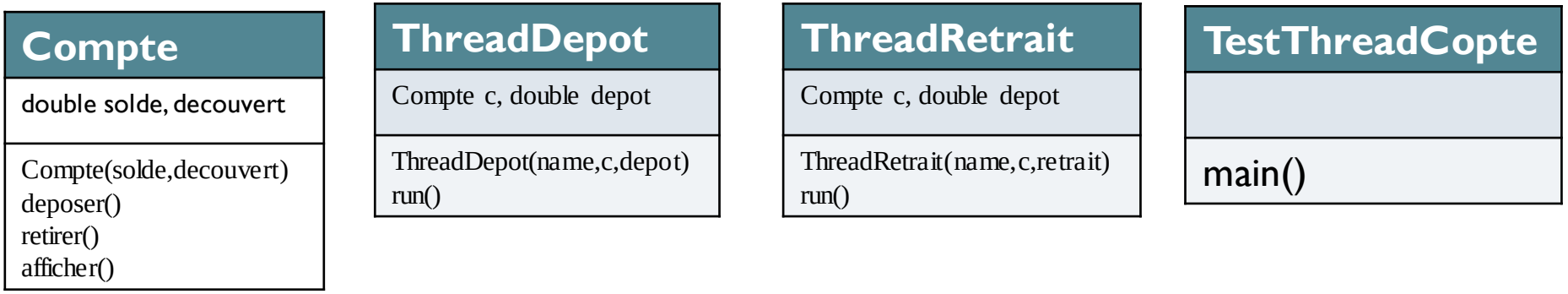
```
class Producteur extends Thread {
MoniteurProdCons tampon;
public Producteur (MoniteurProdCons tampon) { this.tampon=tampon;}
public void run() {
tampon.prod("message1"); tampon.prod("message2");
try {sleep(100);} catch(InterruptedException e) {};
tampon.prod("message3");
}}
class Consommateur extends Thread {
MoniteurProdCons tampon;
public Consommateur(MoniteurProdCons tampon) { this.tampon=tampon;}
public void run() {
tampon.cons(); tampon.cons(); tampon.cons();
}}
```

*Attente explicite : wait - notify

- Le programme principal crée un tampon et une paire de processus producteur/consommateur
- qui communiquent par ce tampon :
- public class TestWait {
- public static void main(String argv[]) {
- MoniteurProdCons tampon = new MoniteurProdCons();
- new Producteur(tampon).start();
- new Consommateur(tampon).start();
- }}

*TRAVAUX PRATIQUES

- Considérons une classe Compte permettant de gérer les comptes de clients dans une quelconque banque disposant de plusieurs agences.
- Pour une gestion efficace et sécurisée des comptes client, il ne faudrait pas permettre par exemple que deux (au moins) transactions s'effectuent simultanément sur un même compte à un instant donné (exemple: le retrait et le dépôt).
- *Les opérations de retrait et de dépôt sont gérées séparément par deux threads distincts.*
- *Votre travail sera de créer ces deux threads de telle sorte qu'ils ne pourront jamais s'exécuter simultanément.*
- *Pour cela créez les classes suivantes:*

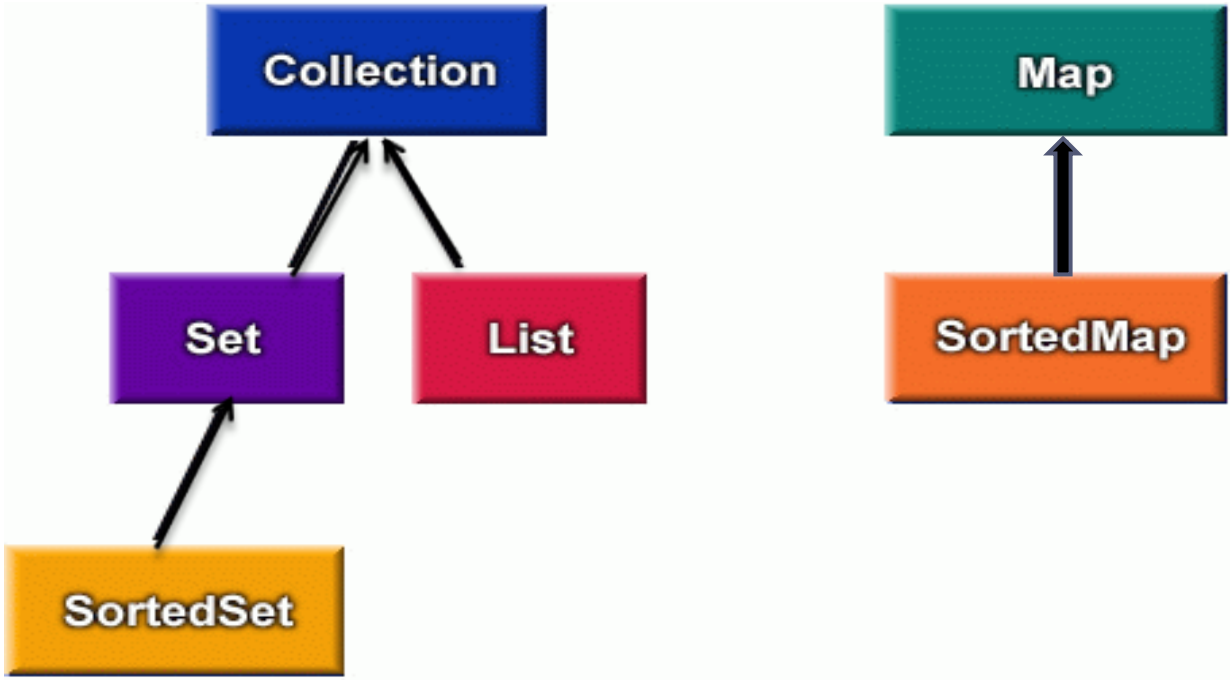


CHAPITRE X

COLLECTIONS

Les Collections

- Une collection est simplement un objet qui regroupe de multiples éléments dans une unité.
- Elle peut stocker des éléments de type quelconque, mais ces éléments sont *obligatoirement des objets.*
- Interfaces:



Les Collections

- Les implémentations des différentes interfaces

	Classes d'implémentations				
		Table de Hachage	Tableau de taille variable	Arbre balancé	Liste chaînée
Interfaces	Set	HashSet		TreeSet	
	List		ArrayList		LinkedList
	Map	HashMap		TreeMap	

L'interface collection

- Collection
- public interface Collection {
- **int size();**//Renvoie le nombre d'éléments de la collection
- **boolean isEmpty();**// Indique si la collection est vide
- **boolean contains(Object element);**//Indique si la collection contient l'élément fourni en paramètre.
- **boolean add(Object element);** //indique si l'élément fourni en paramètre est ajouté
- **boolean remove(Object element);** //Renvoie true si l'élément fourni en paramètre est supprimée.
- **Iterator iterator();**//renvoie un objet qui permet de parcourir l'ensemble des éléments de la collection
- **int hashCode();**//retourne le code de hachage calculé pour la collection.
- **boolean equals(Object element);**//permet de tester si deux objets sont égaux.
- **boolean containsAll(Collection c);**//Si ts les elmts de la collection fournis en paramètre st ds la collect.
- **boolean addAll(Collection c);** //permet d'ajouter des éléments dans la collection
- **boolean removeAll(Collection c);** //Supprime tous les éléments de la collection.
- **boolean retainAll(Collection c);** //intersection. Si A={1,2,5,8} et B={3,8} alors A = {8}.
- **void clear();** //Supprime tous les éléments de la collection
- **Object[] toArray();**Renvoie un tableau d'objet qui contenant les éléments de la collection
- **}**

L'interface iterator

- L'interface collection est dotée d'une instance d'une classe qui implante l'interface Iterator. C'est l'outil utilisé pour parcourir une collection.
- public interface Iterator {
- **boolean hasNext()**;//vérifie s'il y a un élément qui suit.
- **Object next()**;//pointe sur l'élément suivant
- **void remove()**; //retirer l'élément courant
- }

Les differents types de collections

- il existe plusieurs types de collections:
- **Les listes:** Les objets de type List servent à stocker des objets sans condition particulière sur la façon de les stocker. Ils acceptent toutes les valeurs, même les valeurs null.

- **Les types Set:** sont un peu plus restrictifs, car ils n'autorisent pas deux fois la même valeur (le même objet), ce qui est pratique pour une liste d'éléments uniques, par exemple.

- **Les Map:** sont particulières, car elles fonctionnent avec un système clé - valeur pour ranger et retrouver les objets qu'elles contiennent.

L'interface List

- public interface List extends Collection {
- **Object get(int index);** // retourne l'élément se trouvant à la position index.
- **Object set(int index, Object element);** //modifier l'élément se trouvant à la position index.
- **void add(int index, Object element);** // ajouter l'élément se trouvant à la position index.
- **Object remove(int index);** // retire l'élément se trouvant à la position index.
- **boolean addAll(int index, Collection c);** // ajouter les éléments d'une collection se trouvant à la position index.
- **int indexOf(Object o);** //Retourne l'index de l'objet rechercher
- **int lastIndexOf(Object o);**//Retourne le dernier index de l'objet rechercher
- **ListIterator listIterator();**// renvoie un objet pour parcourir la liste

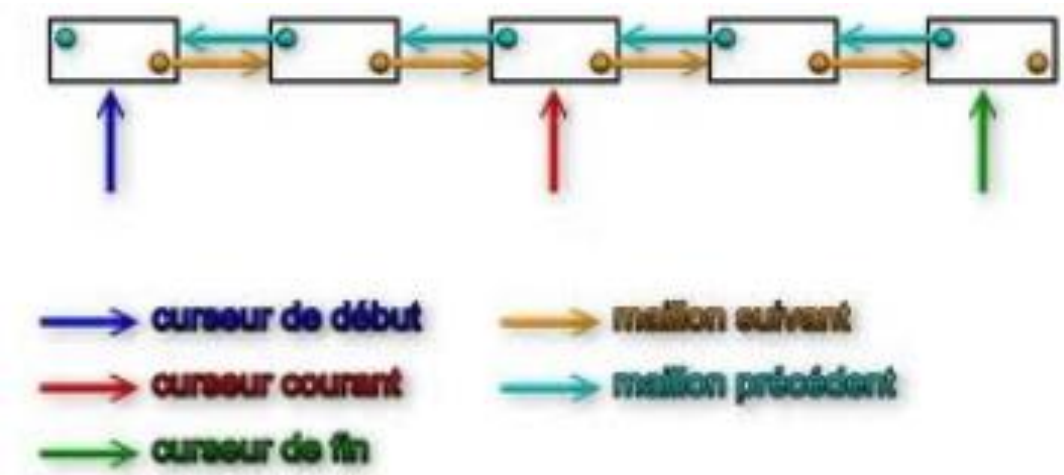
}

L'interface ListIterator

- Pour parcourir une liste, il a été défini un itérateur spécialement pour la liste.
- public interface ListIterator extends Iterator {
- **boolean hasNext()**;//permet de vérifier s'il y a un élément qui suit
- **Object next();**//permet de pointer sur l'élément courant
- **boolean hasPrevious()**;// indique s'il reste au moins un élément à parcourir dans la liste dans son sens inverse
- **Object previous();** //renvoie l'élément précédent dans la liste
- **int nextIndex();**
- **int previousIndex();**
- **void remove();** //
- **void set(Object o);** // remplace l'élément courant par celui en paramètre
- **void add(Object o);** // ajoute un élément dans la liste à la position courante
- }

Les Objets Listes

- Les objets appartenant à la catégorie List sont des tableaux extensibles à volonté. On y trouve les objets **ArrayList et LinkedList.**
- **L'objet LinkedList:**



Exemple avec LinkedList

```
import java.util.LinkedList;
import java.util.List;
import java.util.ListIterator;
public class Test {
  public static void main(String[] args) {
    List l = new LinkedList();
    l.add(12);
    l.add("toto ! !");
    l.add(12.20f);
    for(int i = 0; i < l.size(); i++)
      System.out.println("Élément à l'index " + i + " = " + l.get(i));
  }
}
```

Exemple avec LinkedList(iterateur)

```
import java.util.LinkedList;
import java.util.List;
import java.util.ListIterator;
public class Test {
  public static void main(String[] args) {
    List l = new LinkedList();
    l.add(12);
    l.add("toto ! !");
    l.add(12.20f);
      System.out.println("\n \tParcours avec un itérateur ");
     System.out.println("--------------------------------");
     ListIterator li = l.listIterator();
     while(li.hasNext())
      System.out.println(li.next());
  }}
```

Exemple avec ArrayList

```
public class ListExemple {
public static void main(String args[]) {
List list = new ArrayList();
list.add("Bernadine");
list.add("Elizabeth");
list.add("Gene");
list.add("Elizabeth");
list.add("Clara");
System.out.println(list);
System.out.println("2: " + list.get(2));
System.out.println("0: " + list.get(0));
```

```
LinkedList queue = new LinkedList();
queue.addFirst("Bernadine");
queue.addFirst("Elizabeth");
queue.addFirst("Gene");
queue.addFirst("Elizabeth");
queue.addFirst("Clara");
System.out.println(queue);
queue.removeLast();
queue.removeLast();
System.out.println(queue);}}
```

Les objets MAP

- Une collection de type Map est une collection qui fonctionne avec un couple clé - valeur. On y trouve les objets Hashtable, HashMap, TreeMap.
- Ces objets ont comme point faible majeur leur rapport conflictuel avec la taille des données à stocker. En effet, plus vous aurez de valeurs à mettre dans un objet Map, plus celles-ci seront lentes et lourdes.

L'objet Hashtable

Cet objet nous offre les méthodes suivantes:

isEmpty() retourne « vrai » si l'objet est vide ;

contains(Object value) retourne « vrai » si la valeur est présente.

Identique à containsValue(Object value) ;

containsKey(Object key) retourne « vrai » si la clé passée en paramètre est présente dans Hashtable ;

put(Object key, Object value) ajoute le couple key - value dans l'objet ;

elements() retourne une énumération des éléments de l'objet ;

keys() retourne la liste des clés sous forme d'énumération.

L'objet Hashtable

```
import java.util.Enumeration;
import java.util.Hashtable;
public class Test {
  public static void main(String[] args) {
    Hashtable ht = new Hashtable();
    ht.put(1, "printemps");
    ht.put(10, "été");
    ht.put(12, "automne");
    ht.put(45, "hiver");
    Enumeration e = ht.elements();
    while(e.hasMoreElements())
      System.out.println(e.nextElement());
  }
}
```

L'objet SET

- Un Set est une collection qui n'accepte pas les doublons. Par exemple, elle n'accepte qu'une seule fois null, car deux valeurs null sont considérées comme un doublon. On trouve parmi les Set les objets HashSet, TreeSet, LinkedHashSet

L'objet HashSet

```
import java.util.HashSet;
import java.util.Iterator;

public class Test {
  public static void main(String[] args) {
    HashSet hs = new HashSet();
    hs.add("toto");
    hs.add(12);
    hs.add('d');
    Iterator it = hs.iterator();
    while(it.hasNext())
      System.out.println(it.next());
    System.out.println("\nParcours avec un tableau d'objet");
    System.out.println("--------------------------------");
    Object[] obj = hs.toArray();
    for(Object o : obj)
      System.out.println(o);      }}
```